

1) `S = "python"`

`print(S[0], S[-1], S[6])`

P Y T H O N
(+) 0 1 2 3 4 5
(-) -6 -5 -4 -3 -2 -1

`S[0]` → P

`S[-1]` → N

`S[6]` → out of range

⇒ Error.

2) For Difference B/w positive & negative Index

`S = "HELLO", S[-2] return?`

positive Index starts from (0), left.

Negative Index starts from (-1) right.

`S = "HELLO"`

`S[-2] = "l"`

3) `s = "code"`
`print(s[10])`
`print(s[1:10])`

Indexing:

`code`
 0 1 2 3

`print(s[10])` $\Rightarrow$ Index 10 not exist,

Slicing:

`s = "code"`

`print(s[1:10])`

$\Rightarrow$ "ode". (Note: index starts from 1)

"10" will take as the end of the string.

4)

`s = "MISSISSIPPI"`

$\Rightarrow$ char $\rightarrow$ MISSISSIPPI

Index $\rightarrow$ 0 1 2 3 4 5 6 7 8 9 10

First s = 2, last s = 6.

5) `s = "abcdef"`
`print(s[-1], s[3], s[-6])`

a b c d e f
 -6 -5 -4 -3 -2 -1

$\Rightarrow$ f, d, a.

6) `s = (start : stop : step)`

Start $\rightarrow$ start index

Stop $\rightarrow$ stop index Ending index

Step $\rightarrow$ How many positions to move.
 (increment step)

7) `s = "Hello world"`
 0 1 2 3 4 5 6 7 8 9

`print(s[0:5]) = Hello`

`print(s[5:]) = World`

`print(s[:5]) = Hello`

`print(s[:]) = Hello world`

8) `S = "python"`
`print S[4:1]`

In this step, start should be smaller than stop, so, it returns to blank space.

9) By using slicing we can extract the last 3 char. of a string.

eg: `S = "Danish"`

`print (S[-3:])`

o/p = 'ish'

10) `S = "abcdefgh"`
 0 1 2 3 4 5 6 7

`print (S[::1])` → `abcdefgh`

`print (S[::2])` → `aceg`

`print (S[1::2])` → `bd fh`

11) `s = "python"`
`print (s[::-1])`

o/p $\rightarrow$ nohtyp

The Step Value is (-1).

12) `s = "a b c d e f g h"`

0 1 2 3 4 5 6 7
 -8 -7 -6 -5 -4 -3 -2 -1

`print (s[::-1])` $\rightarrow$ hg fedcba

`print (s[::-2])` $\rightarrow$ hfdb

`print (s[6:1:-1])` $\rightarrow$ gfedc

13) `s = "a 1 b 2 c 3 d 4 e 5"`

`print (s[1::2])`

o/p $\Rightarrow$ "1 2 3 4 5"

14) `s = "python"`

`print (s[5:0:-1])` $\rightarrow$ nohty

`print (s[5::-1])` $\rightarrow$ nohtyp

`print (s[:0:-1])` $\rightarrow$ nohty

15) `s = "hello"`

`print (s[100::-1])` $\rightarrow$ "olleh"

`print (s[::-100])` $\rightarrow$ "o"

16)

`S = "abcdefghij"`

`print (S[2:-1])`

o/p $\rightarrow$ cdefghi

17)

`S = "racecar"`

`S[::-1]` - reverse the string

o/p $\rightarrow$ racecar, It is a palindrome.

True

18)

`S = "Hello"`

`print (S[1:100])` $\rightarrow$ ello

`print (S[-100:3])` $\rightarrow$ Hel

`print (S[100:200])` $\rightarrow$ Blank space

Slicing doesn't raise error when index is out the string range, it adjusts the boundaries.

19) `S = "Hello world"`
`print (S[:6] + "python")`
o/p $\Rightarrow$ Hello python

20) `S = "a b c d e f g h i j"`
0 1 2 3 4 5 6 7 8 9

`My_slice = slice(1, 8, 2)`
`print(S[My_slice])` $\therefore$ `S[1:8:2]`
o/p $\rightarrow$ bdfh.